

Homework 05

Haoran Xu

2026-09-27

All work is in R. Each simulation sets its own seed, shown in the code and restated in the text. KNN uses equal weights and no intercept column.

```
knn_reg <- function(Xtr, ytr, Xte, ks) {  
  # equal-weight Euclidean KNN regression  
  D <- outer(rowSums(Xte^2), rowSums(Xtr^2), "+") - 2 * Xte %*% t(Xtr) # squared distances  
  nb <- t(apply(D, 1, order))[, 1:max(ks), drop = FALSE] # neighbours sorted once, reused  
  # for all k  
  Y <- matrix(ytr[nb], nrow(Xte))  
  sapply(ks, function(k) rowMeans(Y[, 1:k, drop = FALSE]))  
}  
f <- function(X) 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2
```

Question 1 (Estimate the bias and variance of KNN)

Solution

Part a. Seed 43251. $f(x_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.5942$.

```
x0 <- matrix(c(0.5, 0.7, 1), 1); f0 <- f(x0); ks <- seq(1, 29, 2)  
gen1 <- function(n = 400) { X <- matrix(rnorm(n * 3), n); list(X = X, y = f(X) + rnorm(n, sd  
  # = 0.1)) }  
set.seed(43251)  
d <- gen1(); p1 <- as.vector(knn_reg(d$X, d$y, x0, ks))  
c(f_x0 = f0); setNames(round(p1, 3), paste0("k", ks))  
#> f_x0  
#> 0.594218  
#> k1 k3 k5 k7 k9 k11 k13 k15 k17 k19 k21 k23 k25  
#> 0.789 0.662 0.664 0.610 0.632 0.493 0.522 0.581 0.574 0.575 0.580 0.552 0.521  
#> k27 k29  
#> 0.495 0.497
```

The prediction falls from 0.789 ($k = 1$) to about 0.50 ($k = 29$); one data set cannot separate bias from noise.

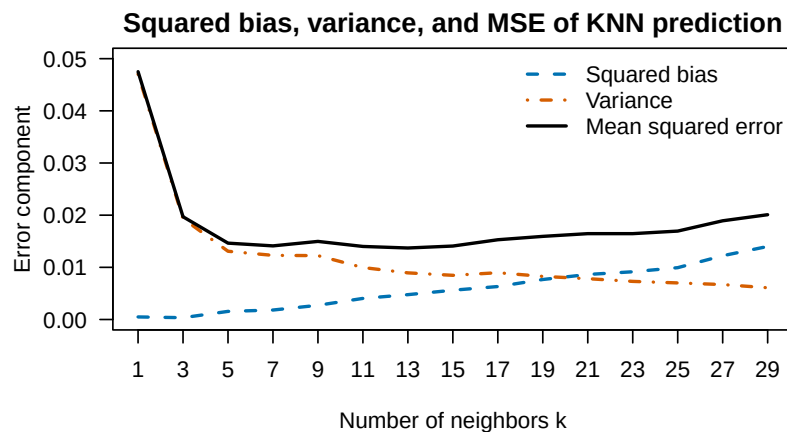
Part b. Seed 43252, 200 independent training sets, the same training set for all 15 values of k .

```
set.seed(43252)  
P <- t(replicate(200, { d <- gen1(); as.vector(knn_reg(d$X, d$y, x0, ks)) }))) # 200 x 15  
bias <- colMeans(P) - f0  
v <- colMeans(sweep(P, 2, colMeans(P))^2) # divisor 200  
mse <- colMeans((P - f0)^2)  
dim(P); max(abs(mse - (bias^2 + v))) # decomposition holds to rounding error  
#> [1] 200 15  
#> [1] 8.32667e-17  
round(rbind(k = ks, bias = bias, bias2 = bias^2, var = v, mse = mse)[, c(1:4, 7, 10, 13,  
  # 15)], 4)  
#> [1,] [2,] [3,] [4,] [5,] [6,] [7,] [8,]  
#> k 1.0000 3.0000 5.0000 7.0000 13.0000 19.0000 25.0000 29.0000
```

```
#> bias   -0.0221 -0.0186 -0.0394 -0.0426 -0.0690 -0.0876 -0.0997 -0.1183
#> bias2   0.0005  0.0003  0.0016  0.0018  0.0048  0.0077  0.0099  0.0140
#> var     0.0470  0.0193  0.0131  0.0123  0.0089  0.0083  0.0070  0.0061
#> mse     0.0475  0.0197  0.0146  0.0141  0.0137  0.0159  0.0169  0.0201
```

The decomposition holds to 8×10^{-17} , i.e. exactly up to rounding.

```
par(mar = c(4, 4, 2, 1), cex = 0.8)
cols <- c("#0072B2", "#D55E00", "#000000"); ltys <- c(2, 4, 1)
matplot(ks, cbind(bias^2, v, mse), type = "l", col = cols, lty = ltys, lwd = 2,
        ylim = c(0, 0.05), xaxt = "n", las = 1,
        xlab = "Number of neighbors k", ylab = "Error component",
        main = "Squared bias, variance, and MSE of KNN prediction")
axis(1, at = ks, las = 1)
legend("topright", legend = c("Squared bias", "Variance", "Mean squared error"),
      col = cols, lty = ltys, lwd = 2, bty = "n")
```



Pattern.

- **Variance** falls with k (0.047 at $k = 1$ to 0.006 at $k = 29$) because more responses are averaged. It exceeds $\sigma^2/k = 0.01/k$ because the neighbour locations are also random.
- **Squared bias** rises with k (about 0 to 0.014) because neighbours lie further from $\mathbf{x}_0$. The bias is negative since f is concave at $\mathbf{x}_0$.
- **MSE** is U-shaped with a flat bottom: minimum 0.0137 at $k = 13$, and $k = 1$ is 3.5 times worse.

Comparison with ridge and lasso.

- Same trade-off: larger k acts like larger λ (less flexible, more bias, less variance).
- Ridge and lasso get bias from shrinking coefficients of a linear model; KNN gets bias from averaging over a region where f changes.
- KNN assumes no form for f , but uses only k local observations, so its variance is higher.

Question 2 (Compare KNN with lasso)

Solution

Part a. Seeds 43253 (Setting 1) and 43254 (Setting 2). `glmnet` is called on covariates I standardize myself (divisor n_{fit}), with a centered response and no internal standardization, so it minimizes exactly the Week 4 objective.

```
library(glmnet); library(MASS)
lam <- 10^seq(-4, 0, length.out = 41)
std <- function(X) { m <- colMeans(X); list(m = m, s = sqrt(colMeans(sweep(X, 2, m)^2))) }
ap <- function(X, p) sweep(sweep(X, 2, p$m), 2, p$s, "/")
lasso <- function(X, y, l) { p <- std(X) # means/scales from the fitting rows
  <- only
```

```

g <- glmnet(ap(X, p), y - mean(y), lambda = sort(1, decreasing = TRUE), standardize =
↪ FALSE,
      intercept = FALSE, thresh = 1e-12, maxit = 1e6)
B <- as.matrix(g$beta)[, order(sort(1, decreasing = TRUE)), drop = FALSE]
list(p = p, b0 = mean(y), B = B) } # unpenalized intercept = mean(y)
plasso <- function(fit, X) fit$b0 + ap(X, fit$p) %*% fit$B
run2 <- function(Sigma, n = 400, nt = 1000) {
  X <- mvrnorm(n, rep(0, 30), Sigma); Xt <- mvrnorm(nt, rep(0, 30), Sigma)
  y <- f(X) + rnorm(n, sd = 0.1); yt <- f(Xt) + rnorm(nt, sd = 0.1)
  fold <- sample(rep(1:10, length.out = n))
  cv <- sapply(1:10, function(k) { i <- fold != k
    colMeans((y[!i] - plasso(lasso(X[i, ], y[i], lam), X[!i, ]))^2) })
  l <- lam[which.min(rowMeans(cv))]; fit <- lasso(X, y, l) # refit on all 400
  list(num = c(knn5 = mean((yt - knn_reg(X, y, Xt, 5))^2), lasso = mean((yt - plasso(fit,
↪ Xt))^2),
      lambda = l, nonzero = sum(fit$B != 0), var_y_test = var(yt)),
    active = which(fit$B != 0))
}
set.seed(43253); r1 <- run2(diag(30))
set.seed(43254); r2 <- run2(0.2 * diag(30) + 0.8)
round(rbind(setting1 = r1$num, setting2 = r2$num), 4); r1$active; r2$active
#>      knn5  lasso lambda nonzero var_y_test
#> setting1 0.5445 0.2262 0.0501      2      0.8283
#> setting2 0.1691 0.2738 0.0200      4      1.4226
#> [1] 1 2
#> [1] 1 2 7 22

```

Table 1: Test MSE on 1,000 test observations; the irreducible error is 0.01.

setting	5NN test MSE	lasso test MSE	selected λ	nonzero slopes
1: independent	0.545	0.226	0.0501	2
2: correlation 0.8	0.169	0.274	0.0200	4

Part b.

- The ranking reverses: lasso wins with independent covariates (0.226 vs 0.545), 5NN wins with correlation 0.8 (0.169 vs 0.274). Lasso barely changes.
- **Independent:** 27 of 30 coordinates are pure noise yet carry equal weight in the distance, so the nearest neighbours are not close in (x_1, x_2, x_3) .
- **Correlated:** all covariates share a common factor, so the data are effectively lower-dimensional and the extra covariates carry information about X_1, X_2, X_3 . Neighbours become informative.
- **Lasso** fits one slope per covariate and sets most to zero, so irrelevant covariates are removed. It kept X_1, X_2 in Setting 1 and X_1, X_2, X_7, X_{22} in Setting 2 (correlated covariates act as proxies). KNN cannot learn that a coordinate is irrelevant.

Part c.

- Lasso is misspecified: a linear fit captures none of $-0.3X_3^2$ (variance 0.18) and only part of $\sin(X_2)$ (residual 0.065), so its MSE cannot go below about 0.25.
- Lasso is already at that floor; its error is mostly bias from the functional form. KNN has no such floor; its error depends on how local its neighbourhoods are.
- So the comparison mixes neighbourhood quality with lasso's linear restriction. With a linear f , lasso would win both settings.
- No general claim is justified. Correlation helped here because it tied the noise covariates to the relevant ones, which need not happen. The winner depends on dimension, correlation, sample size, and nonlinearity, and these are single data sets.

Question 3 (Observed dimension and latent dimension)

Solution

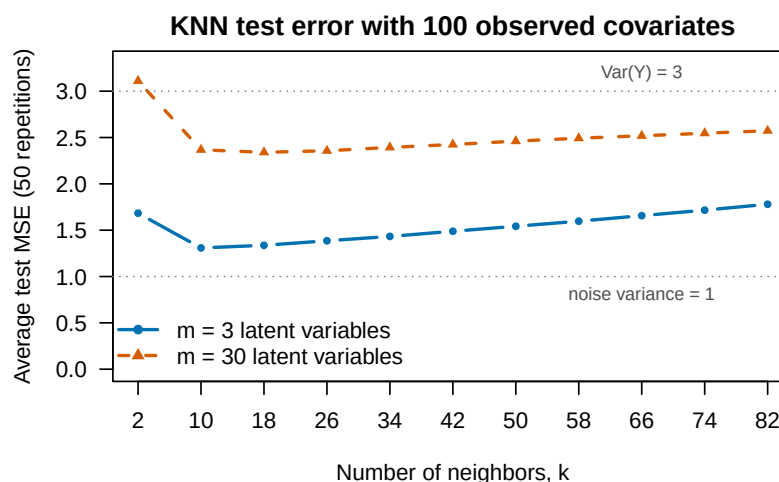
Parts a and b. Seed 43255 for the single data sets and 43256 for the 50 repetitions.

```
ks3 <- seq(2, 82, 8)
run3 <- function(m, n = 200) {
  A <- matrix(runif(100 * m, -2, 2), 100) # generated once per data set
  g <- function() { Z <- matrix(rnorm(n * m), n)
    list(X = Z %*% t(A) + matrix(rnorm(n * 100), n), y = Z[, 1] + Z[, 2] + rnorm(n)) }
  tr <- g(); te <- g() # Z is never given to KNN
  colMeans((te$y - knn_reg(tr$X, tr$y, te$X, ks3))^2)
}

set.seed(43255); one <- rbind(m3 = run3(3), m30 = run3(30))
set.seed(43256); avg <- rbind(m3 = rowMeans(replicate(50, run3(3))),
                             m30 = rowMeans(replicate(50, run3(30))))

colnames(one) <- colnames(avg) <- ks3
round(one, 3); round(avg, 3)
#>      2      10      18      26      34      42      50      58      66      74      82
#> m3  1.692 1.382 1.420 1.483 1.518 1.571 1.606 1.675 1.752 1.827 1.881
#> m30 3.187 2.078 2.045 2.156 2.173 2.164 2.172 2.146 2.168 2.181 2.191
#>      2      10      18      26      34      42      50      58      66      74      82
#> m3  1.683 1.309 1.337 1.385 1.433 1.488 1.542 1.597 1.657 1.716 1.780
#> m30 3.111 2.368 2.340 2.357 2.393 2.425 2.461 2.493 2.518 2.546 2.573

par(mar = c(4, 4, 2, 1), las = 1, cex = 0.8)
matplot(ks3, t(avg), type = "b", lty = c(1, 2), pch = c(16, 17), lwd = 2, cex = 0.8,
        col = c("#0072B2", "#D55E00"), xaxt = "n", ylim = c(0, 3.3),
        xlab = "Number of neighbors, k", ylab = "Average test MSE (50 repetitions)",
        main = "KNN test error with 100 observed covariates")
axis(1, at = ks3); abline(h = c(1, 3), lty = 3, col = "gray50")
text(66, c(1, 3), c("noise variance = 1", "Var(Y) = 3"), pos = c(1, 3), cex = 0.85, col =
  ↪ "gray30")
legend("bottomleft", c("m = 3 latent variables", "m = 30 latent variables"), bty = "n",
      col = c("#0072B2", "#D55E00"), lty = c(1, 2), pch = c(16, 17), lwd = 2)
```



- One data set: best MSE 1.382 ($k = 10$) for $m = 3$ and 2.045 ($k = 18$) for $m = 30$.
- 50 repetitions: minimum 1.309 ($k = 10$) for $m = 3$ and 2.340 ($k = 18$) for $m = 30$.
- Benchmarks: noise variance 1 (best possible) and $\text{Var}(Y) = 3$ (predicting the mean). $m = 3$ achieves about 85% of the possible reduction, $m = 30$ about a third.

Part c.

- **Why performance differs:** the data lie close to an m -dimensional subspace, so KNN effectively faces the latent dimension. 200 points fill 3 dimensions well but are sparse in 30.
- With $m = 30$, 28 latent directions that do not affect Y dominate the distance, so neighbours are not close in (Z_1, Z_2) .
- **Role of η_i :** it perturbs all 100 coordinates and can change which observations are nearest, which is why $m = 3$ reaches 1.31 and not 1.
- **Not confined to the subspace:** $\mathbf{AZ}_i$ lies in the column space of $\mathbf{A}$ (dimension $\leq m$), but η_i has noise in every direction, including those orthogonal to it. The data lie near the subspace, not in it.

Question 4 (Classify handwritten digits)

Solution

ElemStatLearn is not installed on my machine, so I read the course copies of the same `zip.train` / `zip.test` data from `data/knn`. Pixels are used as supplied (range $[-1, 1]$).

Part a.

```
ztr <- as.matrix(read.csv(gzfile("data/knn/zip-train.csv.gz"), header = FALSE))[1:1000, ]
zte <- as.matrix(read.csv(gzfile("data/knn/zip-test.csv.gz"), header = FALSE))[1:500, ]
Xtr <- ztr[, -1]; ytr <- ztr[, 1]; Xte <- zte[, -1]; yte <- zte[, 1]      # label never in a
↪ distance
knn5 <- function(Xtr, ytr, Xte, dist = c("euclidean", "manhattan"), k = 5) {
  dist <- match.arg(dist)
  apply(Xte, 1, function(x0) {
    dif <- sweep(Xtr, 2, x0)      # x_i - x_0 for every training
    ↪ image
    d <- if (dist == "euclidean") sqrt(rowSums(dif^2)) else rowSums(abs(dif))
    nb <- order(d, seq_along(d))[1:k]      # distance ties: smaller row
    ↪ number
    which.max(tabulate(ytr[nb] + 1, nbins = 10)) - 1      # vote ties: smallest digit
  })
}
```

Part b. Rows are predicted digits, columns are true digits.

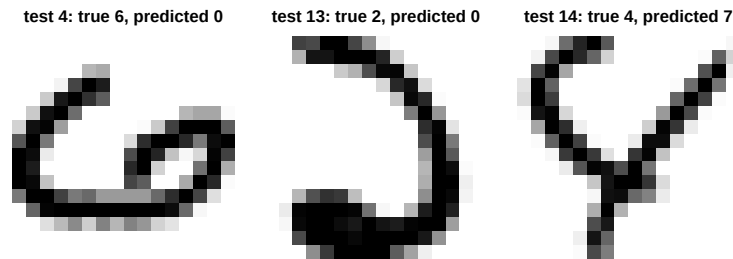
```
pe <- knn5(Xtr, ytr, Xte, "euclidean"); pm <- knn5(Xtr, ytr, Xte, "manhattan")
c(error_euclidean = mean(pe != yte), error_manhattan = mean(pm != yte))
#> error_euclidean error_manhattan
#> 0.122 0.138
table(pred_E = factor(pe, 0:9), true = factor(yte, 0:9))
#> true
#> pred_E 0 1 2 3 4 5 6 7 8 9
#> 0 118 0 7 1 0 4 2 0 2 0
#> 1 0 66 0 0 2 0 0 0 0 1
#> 2 0 0 39 1 0 0 2 0 0 1
#> 3 0 0 0 23 0 4 0 0 2 0
#> 4 0 0 1 0 27 1 1 1 0 1
#> 5 0 0 0 1 0 13 0 0 0 0
#> 6 0 2 0 0 0 1 41 0 2 0
#> 7 0 0 0 0 1 1 0 34 0 4
#> 8 0 0 3 2 0 2 0 0 37 0
#> 9 1 0 0 1 4 0 0 1 1 41
table(pred_M = factor(pm, 0:9), true = factor(yte, 0:9))
#> true
#> pred_M 0 1 2 3 4 5 6 7 8 9
#> 0 118 0 6 1 0 5 3 0 4 0
#> 1 0 67 2 0 2 0 1 0 0 1
#> 2 0 0 35 0 0 0 2 0 0 0
```

```
#>      3  0  0  1  24  0  4  0  0  2  0
#>      4  0  0  1  1  27  1  0  1  0  1
#>      5  0  0  0  1  0  11  0  0  0  0
#>      6  0  1  0  0  0  2  40  0  2  0
#>      7  0  0  2  1  1  0  0  34  0  5
#>      8  0  0  2  0  0  3  0  0  34  0
#>      9  1  0  1  1  4  0  0  1  2  41
```

- Test error: 0.122 (Euclidean, 61 of 500) and 0.138 (Manhattan, 69 of 500).
- Most frequent confusion: true **2 predicted as 0** (7 Euclidean, 6 Manhattan).
- Others: **5 as 0 or 3, 4 as 9, 9 as 7**. Digit 5 is hardest (13 of 26 correct under Euclidean).
- Classes are unbalanced (213 zeros, 47 fives in training), so “0” is the most common wrong prediction.

Part c.

```
w <- which(pe != yte)[1:3]
par(mfrow = c(1, 3), mar = c(0.5, 0.5, 2, 0.5))
for (i in w) { image(t(matrix(Xte[i, ], 16, 16, byrow = TRUE)[16:1, ]), axes = FALSE,
  col = gray(seq(1, 0, length.out = 64)), asp = 1,
  main = sprintf("test %d: true %d, predicted %d", i, yte[i], pe[i]), cex.main = 0.95) }
```



```
both <- pe != yte & pm != yte
table(E_wrong = pe != yte, M_wrong = pm != yte)
#>      M_wrong
#> E_wrong FALSE TRUE
#> FALSE  428  11
#> TRUE    3  58
c(differ = sum(pe != pm), both_wrong_different_label = sum(both & pe != pm))
#>      differ both_wrong_different_label
#>      22 8
```

- **Images** (test rows 4, 13, 14): the 6 lies almost flat with a wide loop like a 0; the 2 is one curved stroke with no flat base; the 4 has no crossbar and looks like a “Y”. The mistakes are understandable: the writing is atypical and the classifier compares pixel positions, not shapes.
- **Overall:** 61 vs 69 errors. The difference (1.6 points on 500 images) is too small to claim Euclidean is better in general.
- **Different mistakes:** yes. Predictions differ on 22 images: 11 wrong only under Manhattan, 3 wrong only under Euclidean, and 8 wrong under both with different labels. 58 images are wrong under both.

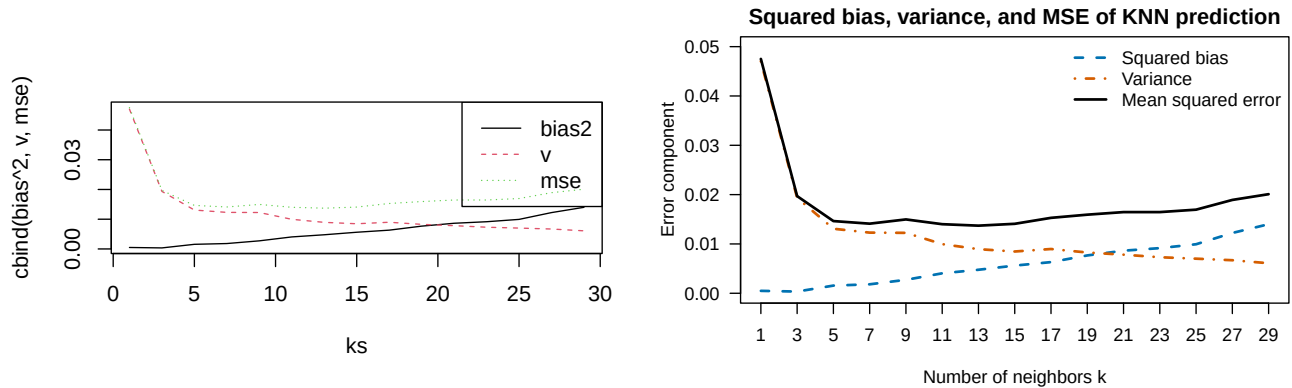
Question 5 (Create a plot-formatting skill)

Solution

Part a. The skill is `stat432-plot-format` (appendix). Its description limits it to STAT 432 homework plots and requires an explicit call by name. Eight portable rules cover margins, title, axes, labels, legend, colors, sizes, and reference lines.

Part b. My agent (Claude Code) installed it in `~/ .claude/skills/stat432-plot-format/`. I called it by name on the Question 1 plot. Left: original. Right: revised.

```
matplot(ks, cbind(bias^2, v, mse), type = "l") # original
legend("topright", c("bias2", "v", "mse"), col = 1:3, lty = 1:3)
```



- **I like the changes:** readable axis labels and legend, a tick at every k , colorblind-safe colors with distinct line types. No data changed.
- **One revision:** the first version added an unrequested reference line and invented a y -axis unit. I tightened those rules and retested; the result is the plot shown.
- I am keeping the skill installed.

Part c.

- **Choices:** short, checkable rules with concrete defaults, and a “formatting only” rule so the skill never touches data or seeds.
- **Experience:** the agent follows rules literally and fills vague wording with extra content, so precise wording matters. It cannot see the rendered figure, so I still check legend placement and text size myself.

Appendix: final SKILL.md

```
---
name: stat432-plot-format
description: Improve the formatting of a plot in a STAT 432 homework report (R or Python).
  ↳ Use only when the student explicitly names "stat432-plot-format" and points to a plot;
  ↳ never trigger it automatically and never use it outside STAT 432 homework.
---

# STAT 432 plot formatting

My intent: make a homework plot easy to read on a printed page without
changing what it shows. Edit formatting only; never alter the data, the
statistics, the random seed, or the conclusions.

## Rules

1. Margins. Trim unused white space: about `mar = c(4, 4, 2, 1)` in base R,
`fig.tight_layout()` or `constrained_layout=True` in matplotlib.
2. Title. One short plain-language title saying what is plotted. No
variable names from the code and no conclusions in the title.
3. Axes. Include zero when the quantity is non-negative with zero as its
natural baseline (errors, variances, counts); otherwise use the data range.
Add about 15% headroom if the legend needs it. Put tick marks at values actually used
  ↳ (e.g. the grid of tuning
values). Use a log axis when the grid is log-spaced. Horizontal tick labels.
```

4. ****Labels.**** Label both axes with the quantity in words or math notation and its unit if it has one (do not invent a unit); never leave a default such as ``x`` or ``V1``.
5. ****Legend and direct labels.**** Every series must be identified, by a legend without a box placed over empty space, or by labels next to the lines.
6. ****Colors.**** At most four colors, from a colorblind-safe set (``#0072B2`` blue, ``#D55E00`` vermillion, ``#009E73`` green, ``#000000`` black). Also vary line type or point shape so the plot survives grayscale printing.
7. ****Sizes.**** Lines about 1.5-2 pt, points visible but not crowded, text no smaller than about 8 pt once the figure is scaled into the report. Size a single panel about 5 x 3 inches and keep the aspect ratio when scaling.
8. ****Reference lines.**** Restyle existing reference lines as thin dashed gray lines. Do not add a reference line the original plot did not have.

Output

Return the revised plotting code only, followed by a short bullet list of the changes, each with the rule number behind it.